

Activation d'abonnement — guide d'intégration mobile

Ce guide couvre deux changements qui vont ensemble :

1. `POST /paiements/:uuid/verifier` — l'application force la vérification du

paiement au lieu d'attendre.

1. **L'abonné est prévenu** dès que son abonnement s'ouvre, par notification

push et par courriel. Le mobile a une donnée à traiter.

Elle répond à un problème mesuré en production : entre le moment où l'utilisateur paie et celui où son abonnement devient actif, il s'écoulait jusqu'à **24 minutes**. Pendant ce temps l'application affiche « en attente » alors que l'argent est encaissé.

1. Pourquoi cette route existe

Le chemin normal est le webhook : le prestataire prévient le backend, qui active l'abonnement en quelques secondes. Quand il fonctionne, l'utilisateur voit son abonnement actif avant même d'avoir rangé son téléphone.

Quand il ne fonctionne pas — endpoint mal déclaré, prestataire muet, réseau coupé — un cron rattrape le paiement **toutes les 30 minutes**. C'est ce rattrapage que vos utilisateurs subissent aujourd'hui.

Mesures relevées en production sur les huit derniers paiements Stripe :

Confirmé par	Délai observé
Webhook	35 s, 37 s, 1 min 41 s
Cron de rattrapage	2 min 17 s → 23 min 47 s

La nouvelle route supprime cette attente : l'application demande elle-même au backend d'interroger le prestataire, et obtient la réponse immédiatement.

Elle ne remplace pas le webhook. Elle le double, pour que l'utilisateur n'attende jamais un mécanisme qu'il ne voit pas.

2. La route

```
POST /paiements/{uuid}/verifier?country=benin
Authorization: Bearer <access_token>
```

Aucun corps de requête. C'est la différence avec `POST /paiements/:uuid/transaction-prestataire`, qui exige une `reference_prestataire` que l'application n'a pas toujours après un retour par redirection. Ici la référence a été enregistrée à l'initiation : le backend la retrouve seul.

Le `uuid` est celui renvoyé par `POST /paiements/initier`.

Réponse — 200

```
{
  "paiement": {
    "uuid": "09a3ea4f-ddd5-42c4-b1d7-7a94d706acab",
    "statut": "REUSSI",
    "montant": 15000,
    "devise": "XOF",
    "prestataire": "STRIPE",
    "date_confirmation": "2026-09-24T07:22:11.284Z"
  },
  "abonnement": {
    "uuid": "5c1f...",
    "statut": "ACTIF",
    "date_debut": "2026-09-24T07:22:11.284Z",
    "date_fin": "2026-10-24T07:22:11.284Z"
  },
  "commande": null
}
```

Un seul appel donne l'état du paiement **et** ce qu'il a débloqué. Ne faites pas un second appel pour lire l'abonnement : sa réponse pourrait arriver avant l'activation et afficher un `EN_ATTENTE` déjà faux.

abonnement et commande s'excluent : un paiement vise soit un abonnement, soit une commande de codes (achat groupé).

Les statuts du paiement

statut	Ce que l'application affiche
INITIE	Paiement créé, pas encore transmis
EN_ATTENTE	En cours chez le prestataire — continuez d'interroger
REUSSI	Encaissé. L'abonnement est actif, ou les codes sont envoyés
ECHOUE	Refusé — proposez de recommencer
ANNULE	Abandonné par l'utilisateur
EXPIRE	Délai dépassé
REMBOURSE	Remboursé

Les quatre derniers sont **définitifs** : n'interrogez plus.

3. Les erreurs

409 — le paiement n'a jamais été transmis

```
{
  "code": "PAIEMENT_NON_TRANSMIS",
  "message": "Ce paiement n'a jamais été transmis au prestataire. Relancez-en un nouveau."
}
```

L'initiation a échoué avant d'atteindre le prestataire : il n'existe nulle part ailleurs que chez nous, et aucune vérification n'est possible. Ce cas n'est pas théorique — la production en compte neuf.

N'insistez pas. Proposez de relancer un paiement.

503 — le prestataire est injoignable

```
{
  "code": "VERIFICATION_INDISPONIBLE",
  "message": "Le prestataire est momentanément injoignable. Réessayez dans un instant."
}
```

Le paiement n'est **pas** en échec : il n'a pas pu être vérifié maintenant. Gardez l'écran d'attente, réessayez plus tard. Le cron repassera de toute façon.

429 — trop d'appels

La route est limitée à **10 appels par minute** et **120 par heure**, par compte. Chaque appel part chez le prestataire ; sans cette limite, une application qui interroge en boucle nous ferait couper par Stripe.

Respectez les intervalles du paragraphe suivant et vous ne la rencontrerez jamais.

404 — paiement introuvable

L'`uuid` n'existe pas, ou appartient à un autre compte. Les deux cas rendent le même message : on ne révèle pas l'existence du paiement d'autrui.

4. Quand appeler

Au retour de la page de paiement, puis en ralentissant :

```
retour → immédiat → 3 s → 5 s → 10 s → 20 s → 30 s → abandon de l'écran
```

Six appels en une minute et demie, bien en deçà de la limite. Si le webhook fonctionne, le premier ou le deuxième suffit.

Arrêtez dès qu'un statut définitif tombe. N'interrogez pas en arrière-plan : si l'utilisateur quitte l'écran, le cron prendra le relais et l'abonnement sera actif à son retour.

5. Exemple Flutter

```
/// Interroge le backend jusqu'à un statut définitif, en espaçant les appels.
/// Renvoie null si l'utilisateur doit être renvoyé vers l'écran d'attente.
Future<EtatPaielement?> suivrePaielement(String uuid) async {
  const attentes = [0, 3, 5, 10, 20, 30];

  for (final secondes in attentes) {
    if (secondes > 0) await Future.delayed(Duration(seconds: secondes));

    final reponse = await http.post(
      Uri.parse('${baseUrl}/paiements/$uuid/verifier?country=$pays'),
      headers: {'Authorization': 'Bearer $token'},
    );

    if (reponse.statusCode == 200) {
      final etat = EtatPaielement.fromJson(jsonDecode(reponse.body));
      if (etat.estDefinitif) return etat; // REUSSI, ECHOUÉ, ANNULE, EXPIRE...
      continue; // EN_ATTENTE : on repasse
    }

    if (reponse.statusCode == 409) {
      // Jamais transmis au prestataire : inutile d'insister.
      final corps = jsonDecode(reponse.body);
      throw PaiementNonTransmis(corps['message']);
    }

    if (reponse.statusCode == 503) {
      continue; // prestataire injoignable, on réessaie au tour suivant
    }

    if (reponse.statusCode == 429) {
      break; // on a été trop gourmand : on laisse le cron faire
    }

    throw Exception('Vérification impossible (${reponse.statusCode})');
  }

  return null; // toujours en attente : écran « paiement en cours »
}
```

Le champ à lire pour débloquent l'écran est `abonnement.statut == "ACTIF"`, pas `paiement.statut == "REUSSI"`. Les deux basculent ensemble, mais c'est l'abonnement qui ouvre l'accès aux ressources.

6. La notification d'activation

Dès qu'un abonnement devient actif, le backend prévient l'abonné sur **deux canaux**, quel que soit le chemin emprunté — paiement encaissé, achat in-app, code d'abonnement offert, ou activation par un administrateur.

Les deux canaux ne font pas double emploi. En production, **un compte sur deux n'a pas de jeton FCM** (18 299 sur 34 475). Sans le courriel, la moitié des abonnés n'apprendraient rien.

La notification push

```
{
  "title": "Votre abonnement est actif",
  "body": "Abonnement mensuel – accès complet jusqu'au 24 octobre 2026.",
  "data": {
    "categorie": "ABONNEMENT_ACTIVE",
    "abonnement_uuid": "c69ef15b-89a2-49cb-ade5-225830ef5507",
    "date_fin": "2026-10-24T07:50:15.489Z"
  }
}
```

Ce que l'application doit faire du champ `data` :

Champ	Usage attendu
categorie	Aiguiller. ABONNEMENT_ACTIVE ouvre l'écran de l'abonnement, pas la liste des notifications
abonnement_uuid	L'abonnement à afficher
date_fin	ISO 8601. Vide ("") si l'abonnement n'a pas d'échéance

Traitez `categorie` comme une valeur ouverte : d'autres viendront. Une valeur inconnue doit ouvrir la liste des notifications, jamais faire planter l'aiguillage.

La notification est aussi **enregistrée en base** : elle apparaît dans `GET /notifications` même si le push s'est perdu, et compte dans le badge de `GET /notifications/unread-count`.

Le jeton FCM doit être à jour

Sans jeton enregistré sur le compte, aucun push n'est tenté — le courriel reste le seul canal. Vérifiez que l'application envoie bien le jeton à la connexion **et à chaque rotation**, sans quoi l'abonné paiera et ne verra rien arriver.

Le courriel

Objet : « Votre abonnement Edukia est actif ». Il rappelle le plan et l'échéance. Rien à faire côté mobile — c'est une trace pour l'abonné.

Ce qui n'est pas annoncé

Une réactivation qui repose l'abonnement en `EXPIRE`, faute de période restante, **ne déclenche rien** : annoncer un accès ouvert serait faux.

Aucun échec d'annonce ne remet l'abonnement en cause. Un abonnement payé reste payé même si la notification et le courriel échouent tous les deux. Ne conditionnez donc **jamais** l'ouverture de l'accès à la réception d'un push : la vérification du paragraphe 2 reste la source de vérité.

7. Ce qui ne change pas

POST <code>/paiements/initier</code>	inchangé
GET <code>/paiements/:uuid</code>	inchangé — lecture simple, sans appel au prestataire
GET <code>/paiements/mes-paiements</code>	inchangé
POST <code>/paiements/:uuid/transaction-prestataire</code>	inchangé — gardez-le si votre SDK fournit une référence

GET `/paiements/:uuid` **ne vérifie rien** : il relit la ligne en base. C'est la confusion la plus probable. Pour forcer une vérification, il faut le `POST ... /verifier`.

8. Rappels

- La route exige un jeton. Le paiement est filtré sur le compte **et** le

pays : un `uuid` d'un autre pays renvoie 404.

- L'appel est idempotent. Sur un paiement déjà définitif, le backend renvoie

l'état sans rappeler le prestataire — vous pouvez le déclencher sans crainte au retour d'un écran.

- Le montant vérifié doit correspondre au centime près. Un écart fait échouer

la confirmation côté serveur, jamais côté application.